

题目 0: 笔记整理

Lecture 05 · Functions 函数

> Advanced Programming — 函数的定义与调用、参数、类型提示、返回值、可变参数、作用域、Lambda 匿名函数

****函数 (Function)****: 可复用的代码块, 用于完成特定任务。作用: 组织代码、避免重复、使程序更易读、易测试、易维护。

****大纲****: ① 定义与调用 ② 参数与实参 ③ 类型提示 ④ 返回值 ⑤ `\*args` / `\*\*kwargs` ⑥ 作用域 ⑦ Lambda

一、定义与调用 (Defining and Calling)

用 `def` 定义, 用括号 `()` 调用。

```
```python
def greet():
 print("Hello!")

greet() # Hello!
```
```

二、参数与实参 (Parameters and Arguments)

- ****形参 (Parameter)****: 定义时括号里的变量

- ****实参 (Argument)****: 调用时实际传入的值

****三种传参方式****

```
```python
1. 位置参数 (Positional): 按位置对应
def add(a, b):
 return a + b
add(2, 3) # 5

2. 关键字参数 (Keyword): 用名字指定, 顺序无所谓
add(a=2, b=3) # 5
add(b=3, a=2) # 5

3. 默认值 (Default): 调用时不传则用默认值
def greet(name="World"):
 print("Hello %s!" % name)
greet() # Hello World!
```
```

```
greet("Bob")          # Hello Bob!
'''
```

> $\triangle$ ****默认参数必须放在非默认参数之后****: `def f(a, b=2)` , `def f(a=1, b)` 

三、类型提示 (Type Hints, 可选但推荐)

提示参数和返回值的类型, ****仅作文档, 运行时不强制检查****。

```
```python
def add(a: int, b: int) -> int:
 return a + b
'''
```

### ## 四、返回值 (Return Values)

- `return` 把值送回调用处并退出函数
- 没有 `return` 的函数默认返回 `None`
- 返回多个值时, 实际打包成 **\*\*元组 (tuple)\*\***, 可直接解包

```
```python
def square(x):
    return x * x
result = square(4)          # 16

def min_max(nums):
    return min(nums), max(nums)    # 实际返回 (min, max)
lo, hi = min_max([3, 1, 4, 1, 5])  # lo=1, hi=5
'''
```

五、可变参数 `\*args` 与 `\*\*kwargs`

`**args` —— **可变位置参数**, 接收为****元组****, 参数个数任意:

```
```python
def total(*args):
 s = 0
 for arg in args:
 s += arg
 return s
total(1, 2, 3) # 6
total(1, 2, 3, 4, 5) # 15
'''
```

**\*\*\*kwargs` —— 可变关键字参数\*\*，接收为\*\*字典\*\*：**

```
```python
def profile(**kwargs):
    for key, value in kwargs.items():
        print(key, "=", value)
profile(name="Alice", age=25, is_female=True)
# name = Alice
# age = 25
# is_female = True
```
```

**\*\*组合使用（课件练习）\*\***

```
```python
def func(a, b=2, *args, **kwargs):
    print(a, b, args, kwargs)

func(1, 3, 4, 5, x=10)
# 输出： 1 3 (4, 5) {'x': 10}
```
```

解析：`a=1`；`b=3`（位置实参 3 **\*\*覆盖\*\***默认值 2）；多余的位置参数 4、5 进入 `args=(4, 5)`；关键字参数 `x=10` 进入 `kwargs={'x': 10}`。

> 四种参数共存时的推荐顺序：`def f(普通参数, 默认参数, \*args, \*\*kwargs)`

## ## 六、作用域：局部 vs 全局（Local vs Global）

函数内部创建的变量是**\*\*局部变量\*\***，函数结束后销毁，不影响全局同名变量。

```
```python
x = 10                                # 全局变量
def change():
    x = 20                            # 局部变量，不影响全局 x
    print(x)
change()                              # 20
print(x)                             # 10（全局 x 未变）
```
```

**\*\*用 `global` 声明可修改全局变量\*\*（通常不推荐，易让数据流难以追踪）：**

```
```python
x = 10
```

```
def change():
    global x          # 声明 x 是全局变量
    x = 20
change()
print(x)              # 20
'''
```

****LEGB 查找规则****: Python 按此顺序查找名字:

****L**ocal** (局部) → ****E**nclosing** (外层嵌套函数) → ****G**lobal** (全局) → ****B**uilt-in** (内置)

```
```python
def outer():
 msg = "enclosing"
 def inner():
 print(msg) # 局部找不到，向外层 (enclosing) 找到
 inner()
outer() # enclosing
'''
```

## ## 七、Lambda 匿名函数

用 `lambda` 关键字定义的小型匿名函数: 可接受任意多个参数, 但函数体只能有**\*\*一个表达式\*\***, 表达式的值自动返回 (无需 `return`)。

**\*\*语法\*\***: `lambda arguments: expression`

```
```python
# 无参数
f = lambda: "hello"
print(f())              # hello

# 多个参数
add = lambda a, b: a + b
print(add(3, 4))        # 7

# 默认参数
greet = lambda name="World": "Hello, %s!" % name
print(greet())           # Hello, World!
print(greet("Alice"))    # Hello, Alice!

# *args
total = lambda *args: sum(args)
```

```

print(total(1, 2, 3))          # 6

# **kwargs
show = lambda **kw: kw
print(show(a=1, b=2))          # {'a': 1, 'b': 2}
...

```

> 典型用途：作为 `sorted(key=...)`、`map()`、`filter()` 等函数的临时小函数；逻辑复杂时仍应使用 `def` 定义普通函数。

题目 1：修改 download 函数

程序如下：

```

# aria2c 下载工具调用程序
# 功能：询问用户下载网址，调用 aria2c.exe 进行下载（整合在 download() 函数中）
# 依赖：aria2c.exe（aria2 1.33.1）

```

```

import subprocess
import os

```

```

ARIA2C = r"e:\Advanced programming\aria2c.exe"    # aria2c.exe 路径

```

```

def download(split=8, max_tries=10):

```

```

    """询问下载网址和保存目录，调用 aria2c.exe 完成下载

```

```

    参数：

```

```

        split      每个文件的分块数（默认 8）

```

```

        max_tries  下载失败时的自动重试次数（默认 10）

```

```

    返回：

```

```

        True  下载成功，False 下载失败或网址为空

```

```

    """

```

```

# 1. 询问下载网址

```

```

url = input("请输入下载网址：").strip()

```

```

if not url:          # 输入为空直接退出

```

```

    print("网址不能为空!")

```

```

    return False

```

```

# 2. 询问保存目录（直接回车保存到当前目录）

```

```

save_dir = input("请输入保存目录（直接回车保存到当前目录）：").strip() or "."

```

```

# 3. 询问分块数与重试次数（直接回车保持默认设置）

```

```

ans = input(f"请输入分块数（直接回车默认 {split}）：").strip()

```

```

if ans.isdigit() and int(ans) > 0:    # 输入合法正整数才更新，否则保持默认

```

```

    split = int(ans)

```

```

ans = input(f'请输入重试次数（直接回车默认 {max_tries}）: ').strip()
if ans.isdigit() and int(ans) > 0:
    max_tries = int(ans)

# 4. 组装命令并调用 aria2c.exe
# -d 指定保存目录
# -s N 每个文件分 N 块下载（分块数）
# -x N 每个服务器最多 N 个连接（与分块数保持一致）
# -m N 下载失败时自动重试 N 次（重试次数）
cmd = [ARIA2C, "-d", save_dir,
        "-s", str(split), "-x", str(split),
        "-m", str(max_tries), url]
print(f'开始下载: {url}')
print(f'保存到: {os.path.abspath(save_dir)}（分块数: {split}, 重试次数: {max_tries}）\n")

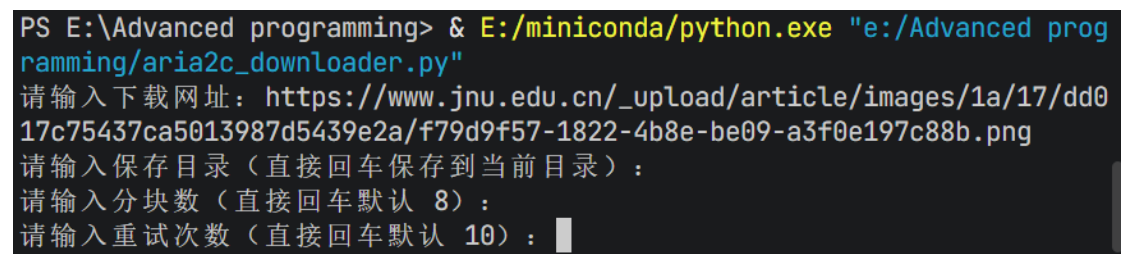
# subprocess.run 调用外部程序，返回码为 0 表示下载成功
result = subprocess.run(cmd)

# 5. 输出结果
if result.returncode == 0:
    print("\n 下载完成!")
    return True
else:
    print("\n 下载失败，请检查网址是否有效。")
    return False

if __name__ == "__main__":
    download()

```

程序运行正常，运行过程如下图：



```

PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced programming/aria2c_downloader.py"
请输入下载网址: https://www.jnu.edu.cn/_upload/article/images/1a/17/dd017c75437ca5013987d5439e2a/f79d9f57-1822-4b8e-be09-a3f0e197c88b.png
请输入保存目录（直接回车保存到当前目录）:
请输入分块数（直接回车默认 8）:
请输入重试次数（直接回车默认 10）:

```

```

PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced programming/aria2c_downloader.py"
请输入下载网址:
https://www.jnu.edu.cn/_upload/article/images/1a/17/dd017c75437ca5013987d5439e2a/f79d9f57-

```

